

Bumbiņu automāts

Madina ir izgudrojusi bumbiņu automātu, lai izklaidētu IOI dalībniekus. Automāta iekšējā uzbūve ir šāda:

- Automāts sastāv no N **virsoņiem**, kas ir sanumurēti no 0 līdz $N - 1$. Virsoņi $N - 1$ saucim par **sakni**.
- Katrai virsoņi u ($0 \leq u < N - 1$) virsoņi u **vecāks** ir virsoņi $P[u]$ ar $P[u] > u$, savukārt virsoņi u ir $P[u]$ **bērns**. Saknei nav vecāka. Virsoņi, kurai nav bērnu, tiek saukti par **lapu**.

Tu **nezini** ne N , ne nevienas virsoņi u vecāku $P[u]$. Bet Madina Tev pasaka lapu skaitu M un to, ka lapu numuri ir $0, 1, \dots, M - 1$. Tavs uzdevums ir noteikt automāta iekšējo struktūru, to darbinot.

Katra automāta virsoņi var saturēt ne vairāk kā vienu **bumbiņu**, un katrai bumbiņai ir **vērtība**, kas ir nenegatīvs vesels skaitlis. Teiksim, ka virsoņi ir vērtība x , ja tajā atrodas bumbiņa ar vērtību x . Virsoņi ir **aizņemta**, ja tajā atrodas bumbiņa; pretējā gadījumā tā ir **brīva**. Sākotnēji visas virsoņi ir brīvas.

Tu vari veikt divu veidu operācijas:

- **insert**(U, X): mēģināt ievietot jaunu bumbiņu ar vērtību X lapā U .
 - Ja lapa U ir aizņemta, šī operācija atgriež `false`, neievietojot bumbiņu.
 - Ja lapa U ir brīva, bumbiņa tiek ievietota virsoņi U . Tad bumbiņa atkārtoti pārvietojas uz savas pašreizējās virsoņi vecāku, kamēr vien tās vecāks ir brīvs. Kustība beidzas, kad bumbiņa sasniedz sakni vai virsoņi, kuras vecāks ir aizņemts. Operācija atgriež `true`.
- **collect**(): ja sakne ir brīva (kas nozīmē, ka automāts ir tukšs), `collect` atgriež tukšu sarakstu. Citādi rekursīvā procedūra `traverse`, kas aprakstīta tālāk, apkopo visu bumbiņu, kas šobrīd atrodas automātā, vērtības masīvā S . Sākotnēji S ir tukšs un tiek izsaukts `traverse(N - 1)`.

```
traverse(u) :
```

```
    pievieno bumbiņas, kas atrodas virsoņi u, vērtību S beigās
```

```
    u aizņemto bērnu sarakstu saucim par c[u]
```

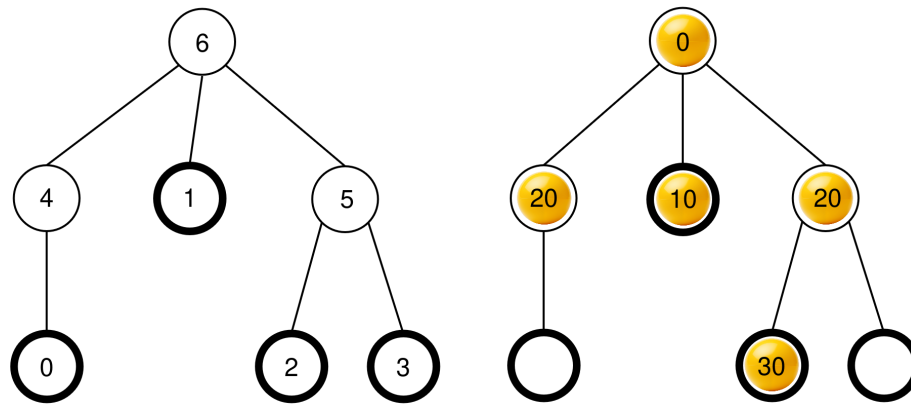
```
    sakārto c[u] nedilstošā secībā pēc bumbiņu, kas atrodas attiecīgajās  
        virsoņi, vērtībām (ja vairākām virsoņi ir tā pati vērtība,  
        tās var parādīties jebkurā secībā)
```

```
    katram v iekš c[u]:
```

```
        traverse(v)
```

Pēc tam, kad šī procedūra beidz darbu, **visas bumbiņas tiek izņemtas** no automāta, un `collect` atgriež S .

Piemēram, aplūkosim automātu ar $N = 7$ virsotnēm un vecāku sarakstu $P = [4, 6, 5, 5, 6, 6]$. Attēlā kreisajā pusē redzams tukšais automāts ar virsotņu numuriem, bet labajā pusē – iespējama bumbiņu konfigurācija pēc noteiktām `insert` operācijām (šo operāciju virkne ir dota sadaļā "Piemēri").



Kad `collect()` tiek izsaukta, sakne (6. virsotne) ir aizņemta, tāpēc S tiek inicializēts kā $S = []$ un `traverse(6)` tiek izsaukta.

`traverse(6)`: 6. virsotnes vērtība ir 0; pievienojot to S , iegūst $S = [0]$. Virsotnes 6 bērni ir 4, 1 un 5, un tie visi ir aizņemti. To vērtības ir attiecīgi 20, 10 un 20. Sakārtojot nedilstošā secībā, iegūst $c[6] = [1, 5, 4]$ (ņemiet vērā, ka $c[6] = [1, 4, 5]$ arī būtu derīgs, jo 4. un 5. virsotnei ir tā pati vērtība). Procedūra tad izsauc `traverse` katrai virsotnei, kas ir iekš $c[6]$, tādā secībā.

- **`traverse(1)`**: 1. virsotnei ir vērtība 10; pievienojot to S , iegūst $S = [0, 10]$. 1. virsotnei nav aizņemtu bērnu, tāpēc šis izsaukums tiek pabeigts.
- **`traverse(5)`**: 5. virsotnei ir vērtība 20; pievienojot to S , iegūst $S = [0, 10, 20]$. 5. virsotnei ir viens aizņemts bērns, 2. virsotne, tāpēc $c[5] = [2]$ un procedūra izsauc `traverse(2)`.
 - **`traverse(2)`**: 2. virsotnei ir vērtība 30; pievienojot to S , iegūst $S = [0, 10, 20, 30]$. 2. virsotnei nav aizņemtu bērnu, tāpēc šis izsaukums tiek pabeigts.
 - Izpilde atgriežas uz `traverse(5)`, kas ir apstrādājusi visus bērnus iekš $c[5]$, tāpēc tās izsaukums beidzas.
- **`traverse(4)`**: 4. virsotnei ir vērtība 20; pievienojot to S , iegūstam $S = [0, 10, 20, 30, 20]$. 4. virsotnei nav aizņemtu bērnu, tāpēc tās izsaukums tiek pabeigts.

Visi izsaukumi ir pabeigti un vairs nav apstrādājamu virsotņu. Visbeidzot visas bumbiņas tiek izņemtas no automāta un `collect` atgriež sarakstu $S = [0, 10, 20, 30, 20]$.

Tavs uzdevums ir noteikt virsotņu skaitu N un atgriezt vecāku sarakstu $R = [R[0], R[1], \dots, R[N - 2]]$, kas raksturo automāta struktūru, bet, iespējams, ar citiem virsotņu,

kas nav ne lapas, ne sakne, numuriem. Formāli atgrieztais saraksts R tiek uzskatīts par pareizu, ja ir iespējams katrai automāta virsotnei u piešķirt atšķirīgu numuru $L[u]$, kur $0 \leq L[u] < N$, tā, ka:

- $L[u] = u$ visiem $0 \leq u < M$ un $u = N - 1$, un
- $L[P[u]] = R[L[u]]$ visiem $0 \leq u < N - 1$.

$R[u] > u$ **nav** obligāti jāizpildās. Automātam **ir jābūt tukšam**, kad Tu atgriez savu risinājumu.

Izpildīto `collect` operāciju skaitu apzīmēsim ar K , un maksimālo bumbiņas vērtību, kas parādās kādā `insert` izsaukumā, apzīmēsim ar B . Apzīmēsim $C = K + B$. Tad C **nedrīkst pārsniegt** 1000. Dažos apakšuzdevumos Tavs punktu skaits ir atkarīgs no C vērtības.

Implementēšanas detaļas

Tev jāimplementē šāda procedūra.

```
std::vector<int> find_structure(int M)
```

- M : bumbiņu automāta lapu skaits.
- Procedūra katram testam tiks izsaukta tieši vienu reizi.
- Procedūrai jāatgriež masīvs $R = [R[0], R[1], \dots, R[N - 2]]$ no $N - 1$ skaitļiem – pareizs vecāku saraksts.

Lai mijiedarbotos ar automātu, Tava procedūra var izsaukt šādas divas procedūras.

Pirmā procedūra ir:

```
bool insert(int U, int X)
```

- U : lapa, kurā vajadzētu ievietot bumbiņu, numurs. Jāizpildās $0 \leq U < M$.
- X : vesels skaitlis, bumbiņas vērtība. Jāizpildās $0 \leq X \leq 1000$.
- Šī procedūra atgriež `true`, ja bumbiņa tika veiksmīgi ievietota, vai `false`, ja lapa U jau bija aizņemta.
- Šī procedūra var tikt izsaukta ne vairāk kā 500 000 reizes.

Otrā procedūra ir:

```
std::vector<int> collect()
```

- Apkopo visu ievietoto bumbiņu vērtības, iztukšo automātu, un atgriež apkopoto sarakstu S .

Ja kādā brīdī Tavas programmas izpildes laikā C vērtība pārsniedz 1000, Tavs risinājums saņems vērtējumu `Output isn't correct: Too many resources used` (Izvaddati nav pareizi: Izmantots pārāk daudz resursu).

Bumbiņu automāta struktūra ir fiksēta pirms `find_structure` tiek izsaukta. Vērtētāja programma ir **deterministiska** tādā nozīmē, ka, ja to izpilda divas reizes, un abos gadījumos tiek izpildītas tās pašas operācijas, `collect` izsaukumi atgriezīs tos pašus masīvus.

Ierobežojumi

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Apakšuzdevumi

Apakšuzdevums	Punkti	Papildu ierobežojumi
1	5	Saknei ir tieši M bērni.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Bez papildu ierobežojumiem.

Ņem vērā, ka 3. un 4. apakšuzdevumā Tavi punkti ir atkarīgi no C vērtības.

3. apakšuzdevums (25 punkti)

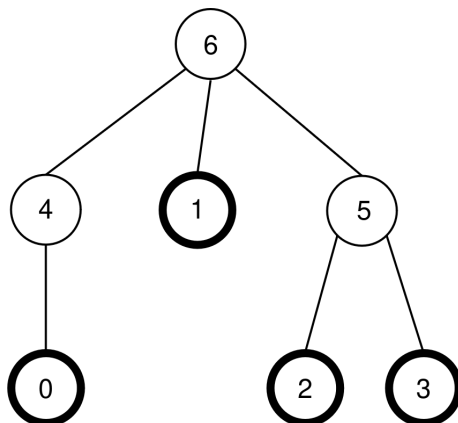
Ierobežojumi	Punkti
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

4. apakšuzdevums (60 punkti)

Ierobežojumi	Punkti
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Piemērs

Aplūkosim tādu pašu bumbiņu automātu kā uzdevuma aprakstā, tas ir, $N = 7$, $M = 4$ un $P = [4, 6, 5, 5, 6, 6]$. Automāts ir redzams attēlā:



Vērtētājs veic šādu izsaukumu:

```
find_structure(4)
```

Procedūra var veikt šādus izsaukumus:

1. **insert(0, 0)**: lapa 0 ir brīva, tāpēc bumbiņa ar vērtību 0 tiek ievietota lapā 0. Virsotne $P[0] = 4$ ir brīva, tāpēc bumbiņa pārvietojas uz virsotni 4. Virsotne $P[4] = 6$ ir brīva, tāpēc bumbiņa pārvietojas uz virsotni 6. Tā kā virsotne 6 ir sakne, tad kustība beidzas. Izsaukums atgriež `true`.
2. **insert(3, 20)**: lapa 3 ir brīva, tāpēc bumbiņa ar vērtību 20 tiek ievietota lapā 3. Virsotne $P[3] = 5$ ir brīva, tāpēc bumbiņa pārvietojas uz virsotni 5. Tā kā $P[5] = 6$ ir aizņemta, tad kustība beidzas. Izsaukums atgriež `true`.
3. **insert(1, 10)**: lapa 1 ir brīva, tāpēc bumbiņa ar vērtību 10 tiek ievietota lapā 1. Tā kā $P[1] = 6$ ir aizņemta, bumbiņa paliek virsotnē 1. Izsaukums atgriež `true`.
4. **insert(2, 30)**: lapa 2 ir brīva, tāpēc bumbiņa ar vērtību 30 tiek ievietota lapā 2. Tā kā $P[2] = 5$ ir aizņemta, bumbiņa paliek virsotnē 2. Izsaukums atgriež `true`.
5. **insert(1, 25)**: lapa 1 ir aizņemta, tāpēc bumbiņa netiek ievietota lapā 1. Izsaukums atgriež `false`.
6. **insert(0, 20)**: lapa 0 ir brīva, tāpēc bumbiņa ar vērtību 20 tiek ievietota lapā 0. Virsotne $P[0] = 4$ ir brīva, tāpēc bumbiņa pārvietojas uz virsotni 4. Tā kā $P[4] = 6$ ir aizņemta, kustība beidzas. Izsaukums atgriež `true`.

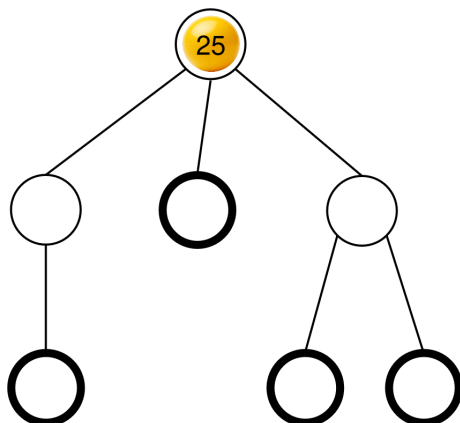
Iegūtais bumbiņu izkārtojums jau tika parādīts uzdevuma aprakstā.

Pēc tam procedūra var izsaukt:

```
collect()
```

Šī izsaukuma izpilde jau tika paskaidrota uzdevuma aprakstā. Izsaukums atgriež $S = [0, 10, 20, 30, 20]$. Pēc tam bumbiņu automāts atkal ir tukšs.

Pēc tam procedūra var izsaukt `insert(2, 25)`. Lapa 2 ir brīva, tāpēc bumbiņa ar vērtību 25 tiek ievietota 2. lapā. Tad šī bumbiņa pārvietojas uz 5. virsotni, un tad uz 6. virsotni, kur tā apstājas. Izsaukums atgriež `true`. Iegūtais bumbiņu izkārtojums redzams attēlā.



Visbeidzot procedūra var izsaukt `collect()`, ka atgriež $S = [25]$ un iztukšo automātu.

Ir divi pareizi vecāku masīvi, ko `find_structure` var atgriezt: $R = P = [4, 6, 5, 5, 6, 6]$ (kas atbilst numuriem $L = [0, 1, 2, 3, 4, 5, 6]$), un $R = [5, 6, 4, 4, 6, 6]$ (kas atbilst numuriem $L = [0, 1, 2, 3, 5, 4, 6]$). Šajā piemērā $K = 2$ un $B = 30$, tāpēc $C = 32$.

Paraugvērtētājs

Ievaddatu formāts:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Izvaddatu formāts:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Šeit Q ir masīva R garums, ko atgriež `find_structure`.